

RecruitMe

Project Scope · Current State · Future Implementations · Running Costs

Prepared 4 June 2026 · Figures from live production data where measured; estimates flagged.

1 · What it is

RecruitMe is a recruiter-facing SaaS that sources, scores, and manages candidates for open roles. A recruiter creates a **job** (pastes a job description → AI parses it into structured must-haves, nice-to-haves, seniority, location and salary), then **finds candidates** from three sources — an internal **talent pool** (~15k profiles), **LinkedIn**, and **SEEK Talent Search** — and the system **AI-scores** each candidate against the role, producing a ranked shortlist with a written rationale per candidate.

Multi-tenant (organisation-scoped), single primary operator today. Deployed on Railway; live candidate discovery runs through a self-hosted mini-PC worker.

2 · Current state

The platform holds ~**15,000 candidate profiles** (CVs encrypted at rest), kept current via a **live JobAdder feed** and grown further through LinkedIn and SEEK discovery.

Live JOBADDER FEED	808 AI-SCORED	9 ACTIVE JOBS	3 CANDIDATE SOURCES
------------------------------	-------------------------	-------------------------	-------------------------------

Technology

- **App:** Next.js 15 / React 19 / TypeScript, Prisma ORM, PostgreSQL 18 — hosted on **Railway** (auto-deploy from GitHub `main`).
- **AI:** Anthropic Claude — **Haiku 4.5** for JD parsing & snippet scoring (cheap), **Sonnet 4.6** for full-profile scoring (quality). Cost-tracked per call with a daily spend cap.
- **Discovery worker:** self-hosted mini-PC ("the box") running a persistent logged-in browser for LinkedIn + SEEK; reached over Tailscale. Single-worker by design.
- **Storage:** CVs in an S3-compatible bucket, **AES-256-GCM encrypted at rest**, original sender format preserved.

Shipped & working

- **JD analysis** — Claude parses a pasted JD into structured requirements, search expansion and title variants.
- **Multi-source search** — internal pool (Postgres full-text + AI ranking), live LinkedIn discovery, live SEEK Talent Search. Library-first: cheap pool results instantly, live sources on demand.
- **AI scoring** — per-candidate match score (0–100) with must-have coverage, category breakdown and recruiter summary; provisional vs full-profile scoring; per-org configurable weights; cached so unchanged candidates aren't re-billed.
- **SEEK Talent Search** — ingests result cards as candidates, credit-safe (free card data, no per-profile credit spend). Validated 4 Jun 2026.

- **Live JobAdder feed** — the operator's JobAdder candidate data (~13k profiles) is available live and on demand, drawn from the operator's own JobAdder account, and kept current in the platform.
- **Talent pool / flywheel** — every search enriches the pool; future searches serve those candidates instantly.
- **Candidate management** — library, per-job pipelines, CV upload + text extraction, identity dedup across sources, screening data, recruiter notes, CSV export.
- **Operations** — on-box health dashboard (CPU/RAM/disk/temp, worker status), cost attribution + daily spend cap, PII redaction, CI real-DB test gate, error reporting.

Known constraints / technical debt

- **Two parallel search systems** (legacy job-search vs newer multi-source) not yet unified — see roadmap.
- SEEK candidates are **snippet-level**; full-profile enrichment is a separate, credit-gated action.
- ~15k candidates, **808 scored** — large unscored backlog (scoring is on-demand + cost-capped).
- Single discovery worker (no redundancy); the box must stay online for live discovery.

3 · Future implementations

Initiative	Description
A. Search consolidation	Collapse the two parallel search systems into one federated dispatcher + pluggable source adapters (pool / LinkedIn / SEEK / future), one durable result model, and a live-streaming results UI (results appear as they arrive). Removes current fragility; makes adding a source trivial. ~6 phases, each independently shippable. unblocks most future work
B. SEEK enrichment	One-click, credit-aware deep-fetch of a chosen SEEK candidate's full profile (capability exists; needs wiring + a credit guard).
C. Scoring at scale	Score the unscored backlog; background/batch scoring; richer re-score-on-change.
D. Talent flywheel	Background discovery that continuously enriches the pool (cost-gated); profile freshness / re-fetch of stale profiles.
E. Candidate UX	Largely shipped (email/screening/file surfacing, import-batch grouping, source badges); remaining refinements.
F. Hardening	Worker auth hardening; multi-worker redundancy if discovery volume grows; deeper observability.

4 · Running costs per month

Item	\$ / month	Basis
Claude API (Anthropic)	~\$15–40	Measured: \$13.37 over the last 30 days (865 calls, ~3.7M tokens) — mostly cheap Haiku. With Sonnet now scoring full profiles (~\$0.05/score) cost scales with scoring volume; hard-capped at \$5/day (≈ \$150/mo ceiling).

Railway (app + Postgres + egress)	~\$15–25	Hobby plan (\$5 base + metered): always-on app + Postgres + bandwidth. Estimate.
CV storage (encrypted S3)	~\$1–3	~13.5k encrypted CVs (~8–12 GB) + light egress. Estimate.
Mini-PC worker (electricity)	~\$3–6	Low-power i3 mini-PC 24/7 at NZ rates. Internet = existing home line; Tailscale = free. Estimate.
Domain	\$0	Railway subdomain (custom domain would add ~\$1–2/mo).
Platform infrastructure total	~\$35–65	Everything RecruitMe itself costs to run.
<i>SEEK Talent Search</i>	<i>operator's sub</i>	<i>External recruiting tool, the operator's existing SEEK account. Per-search/profile credits apply — the app is built to be credit-safe.</i>
<i>LinkedIn</i>	<i>operator's acct</i>	<i>Uses the operator's existing LinkedIn; no extra app cost.</i>
<i>JobAdder</i>	<i>operator's sub</i>	<i>The operator's existing JobAdder account; the live feed draws that data into the platform.</i>

Bottom line: the RecruitMe *platform* runs at roughly **\$35–65 / month** in direct infrastructure + AI cost, on top of the operator's existing SEEK / LinkedIn / JobAdder subscriptions. The single largest and most variable line is the Claude API, which is bounded by the daily spend cap and scales with how much scoring is done.

5 • Tech-stack summary (developer handoff)

- **Frontend / Backend:** Next.js 15 (App Router), React 19, TypeScript, Tailwind.
- **Data:** PostgreSQL 18 + Prisma; Postgres full-text search for the talent pool.
- **AI:** Anthropic SDK (Claude Haiku 4.5 + Sonnet 4.6); structured scoring pipeline with caching, cost tracking and spend caps.
- **Discovery:** Node/TypeScript worker on a mini-PC, patched Chromium, one persistent session per platform; polls a Postgres-backed job queue (priority + dedup).
- **Infra:** Railway (CI real-DB gate on deploy), Tailscale (box networking), S3-compatible blob storage (encrypted CVs).
- **Repo:** GitHub `baeley-canning/RecruitMe` — deploy = push to `main`.